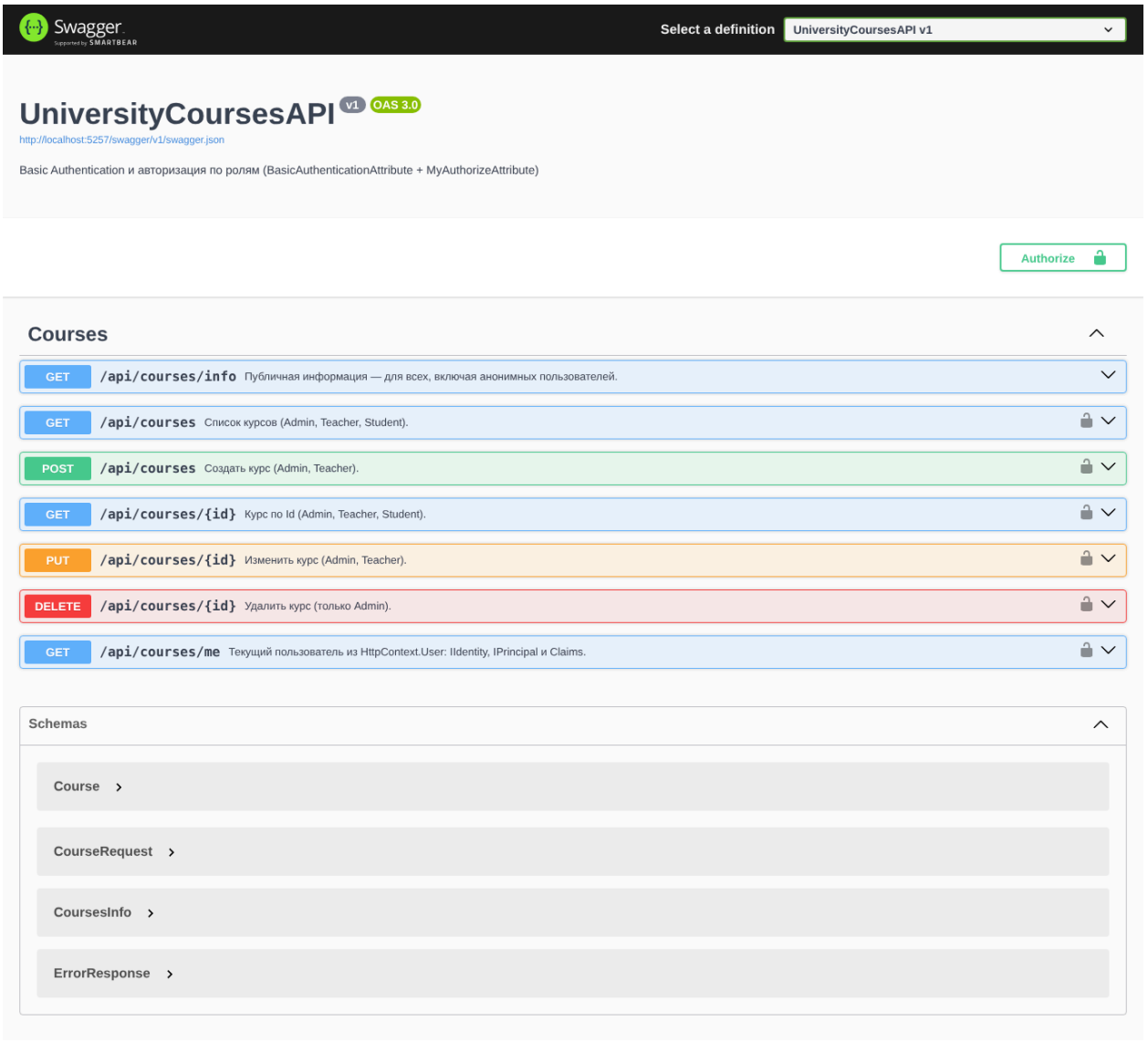
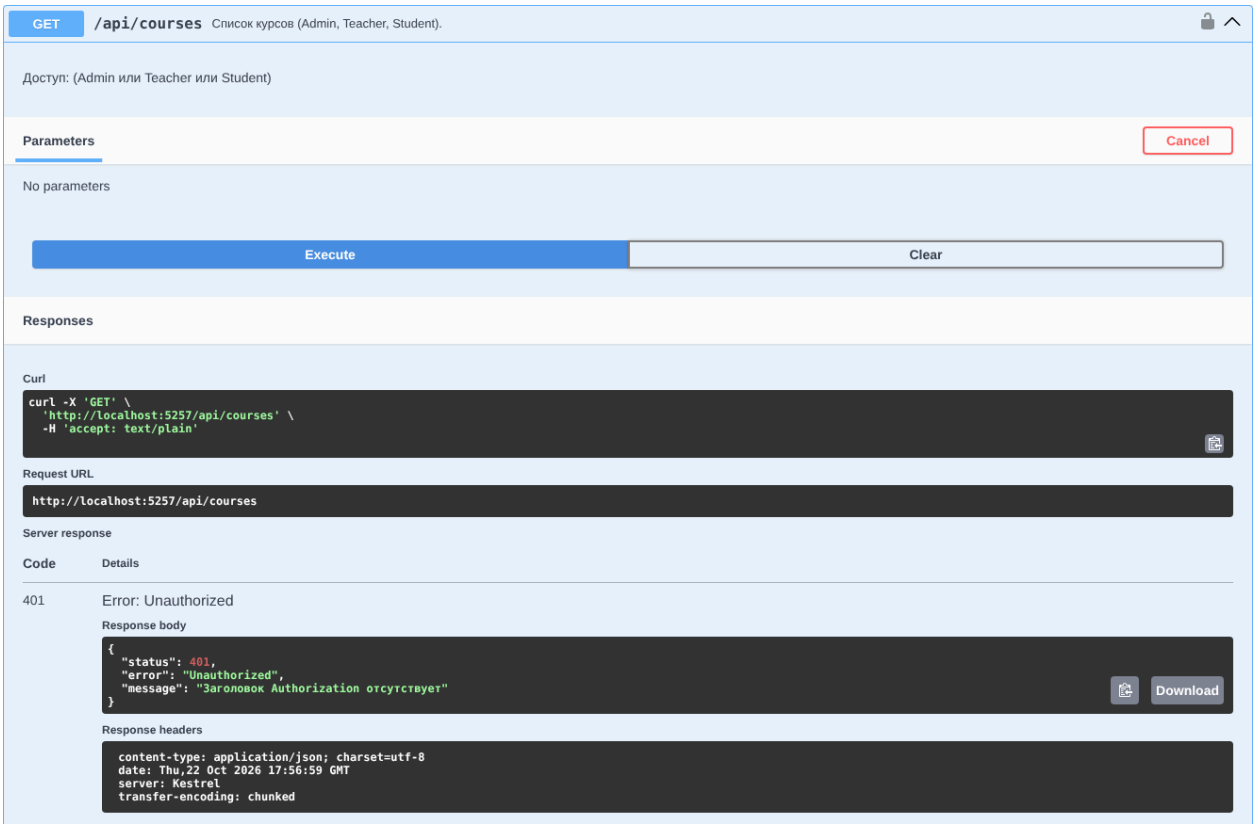
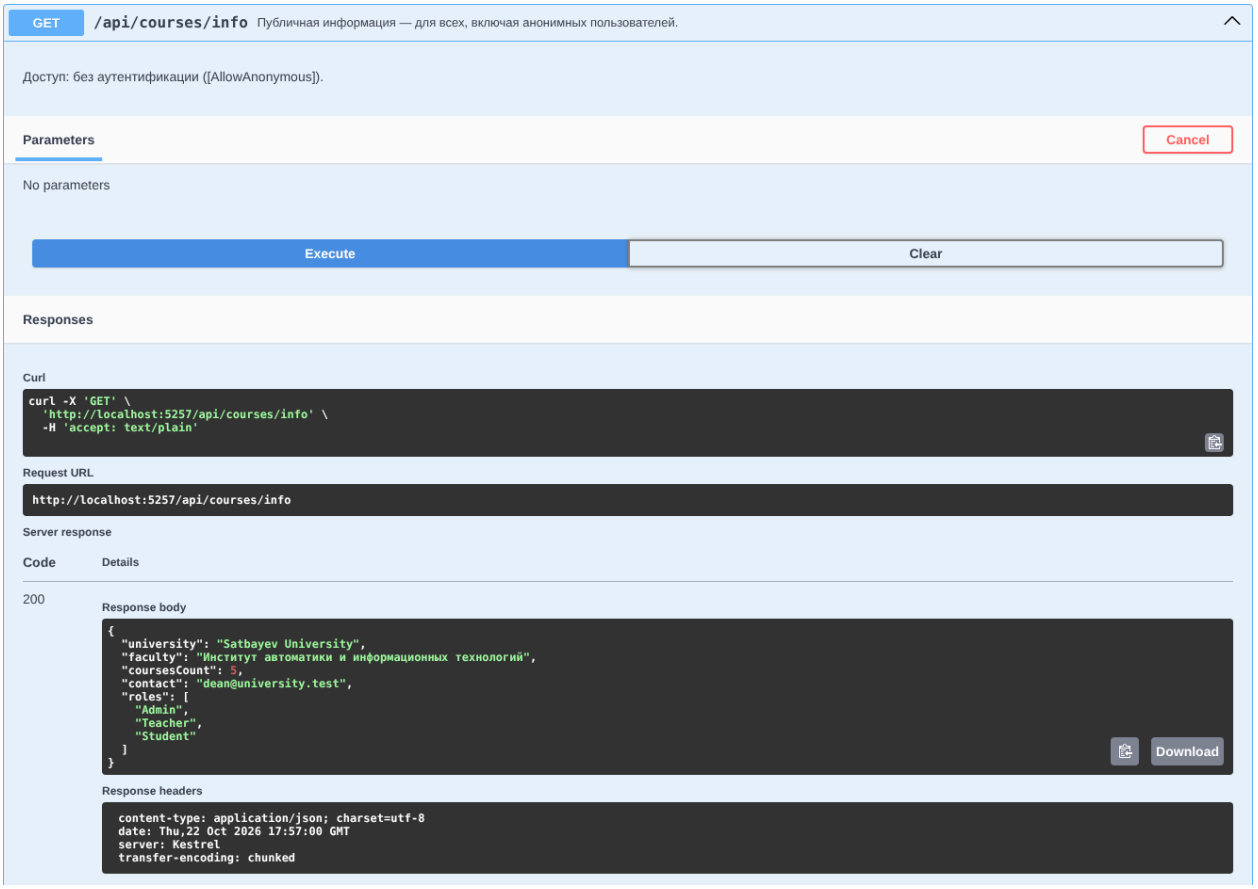
Домашняя работа 7



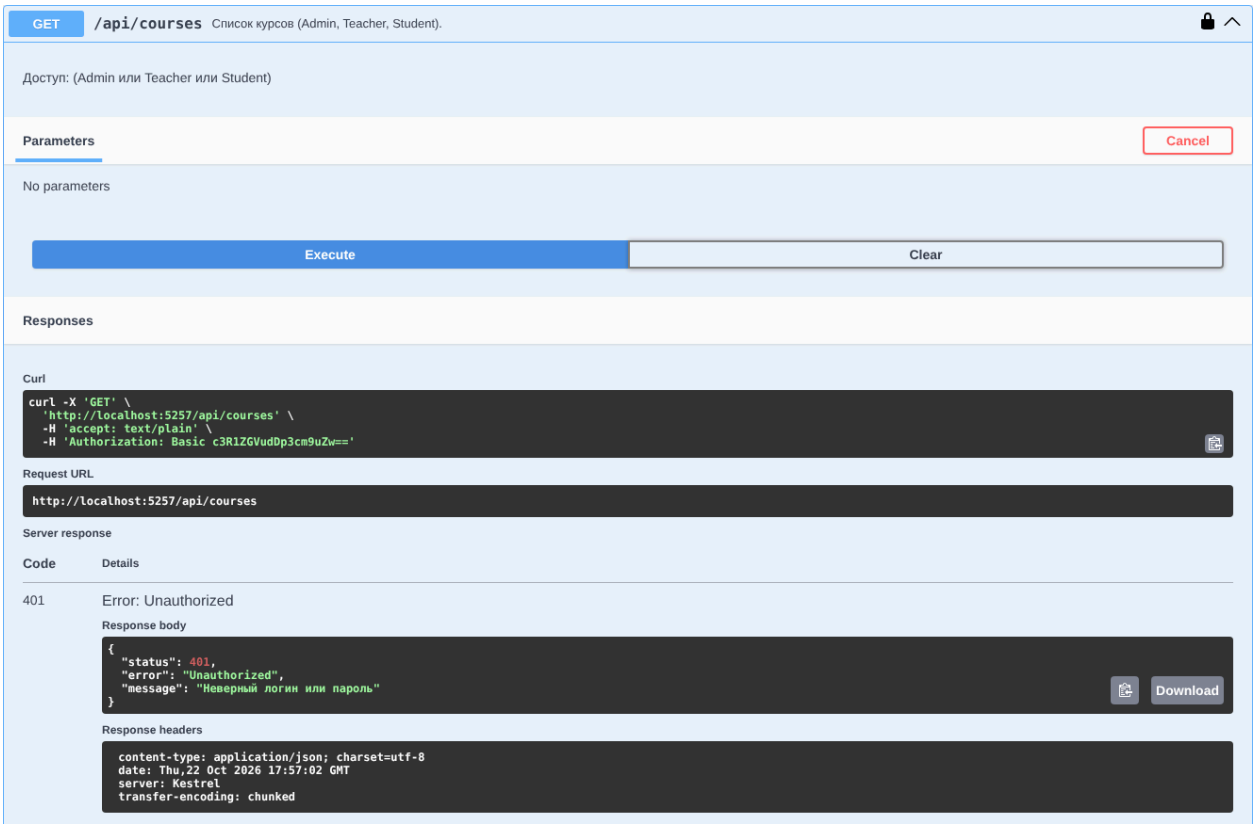
1 - Swagger UI UniversityCoursesAPI: /api/courses/info без замка, остальные методы защищены



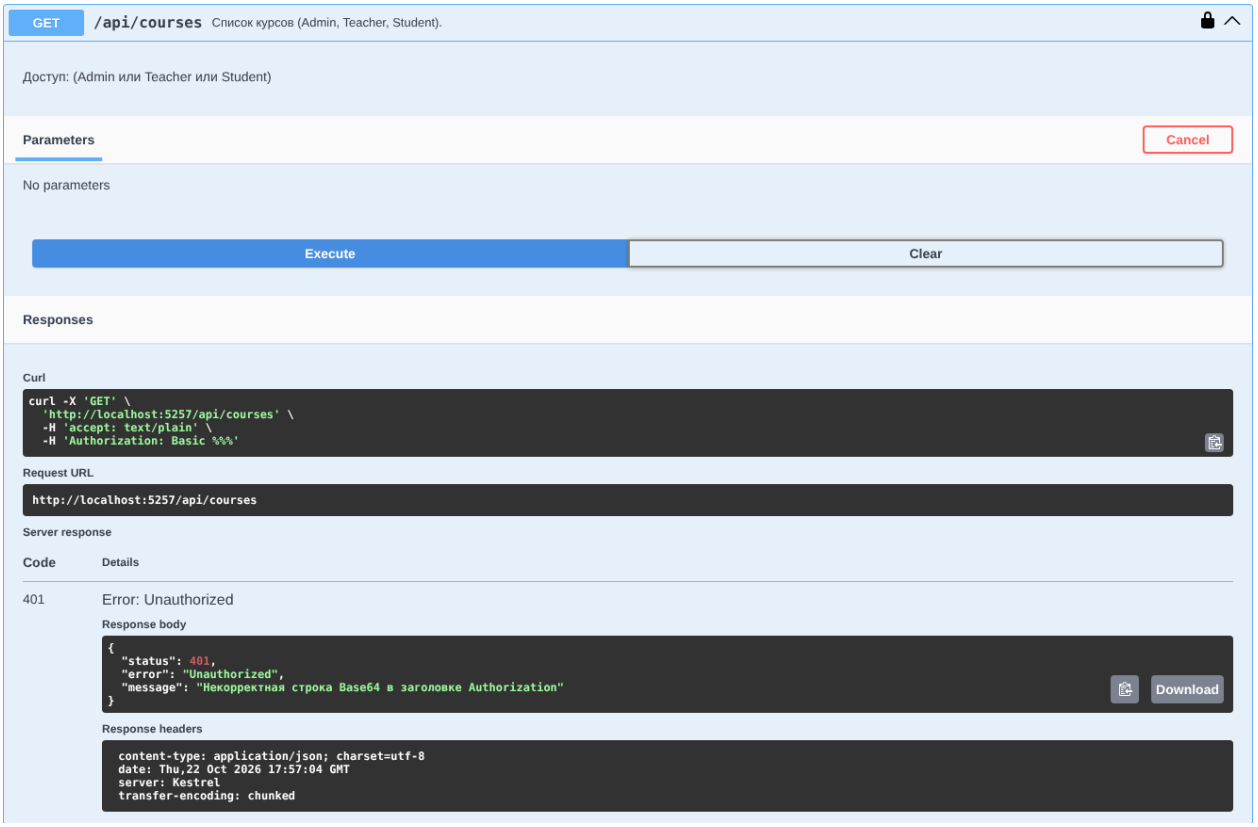
2 - Сценарий 1. Список курсов без аутентификации - 401 Unauthorized



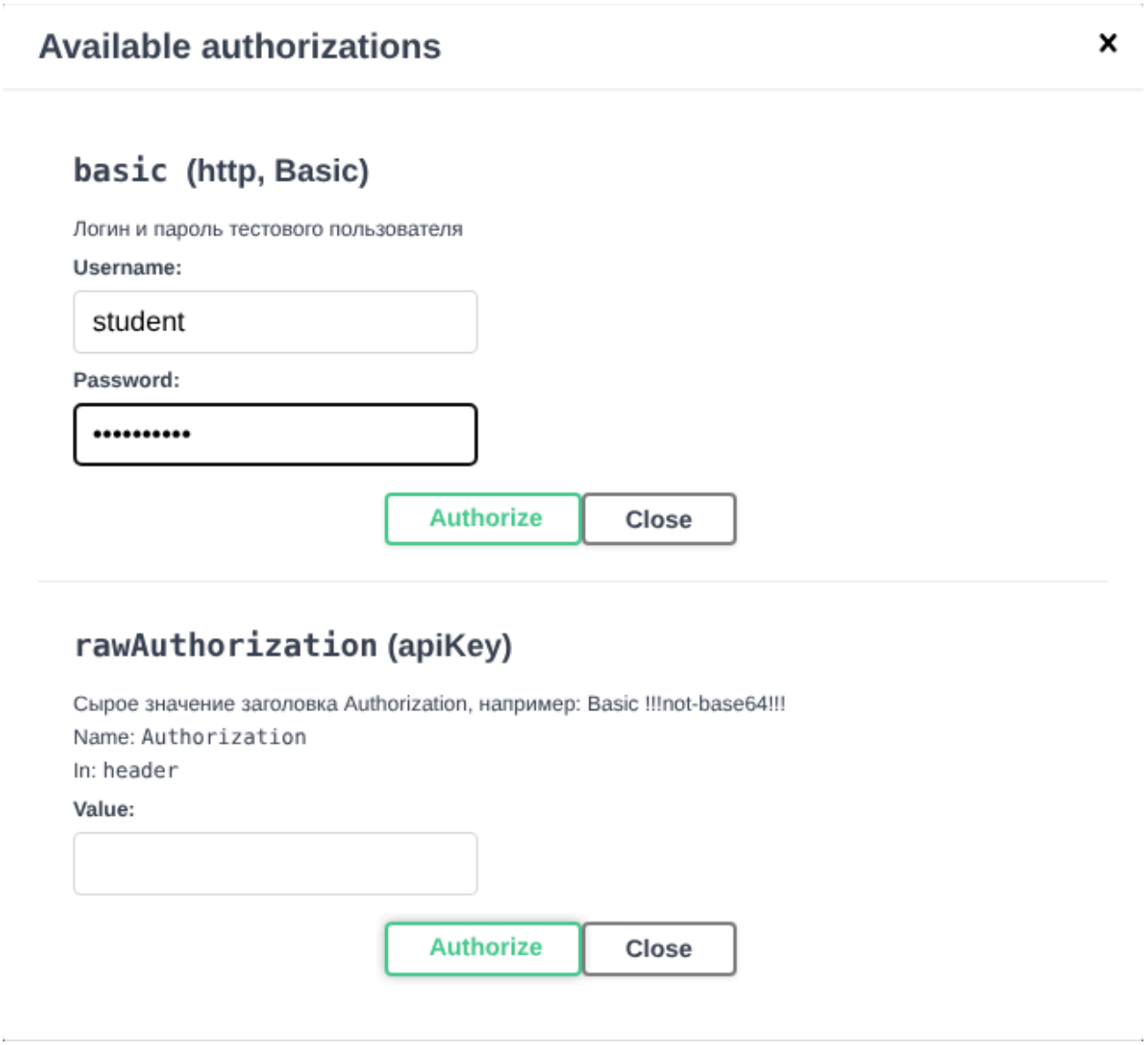
3 - Сценарий 8. Публичная информация без аутентификации - 200 OK



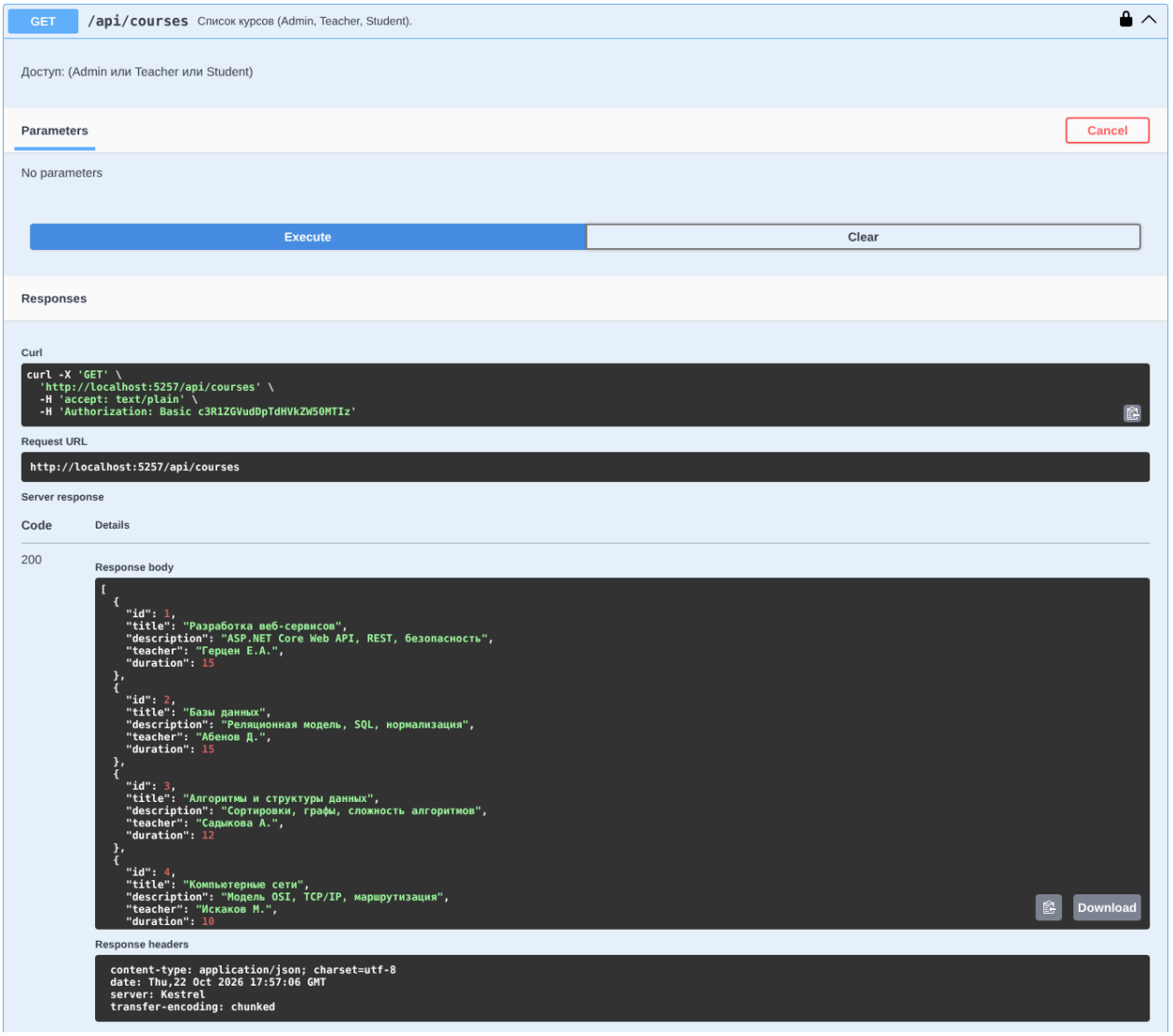
4 - Сценарий 7. Неправильный пароль - 401 Unauthorized



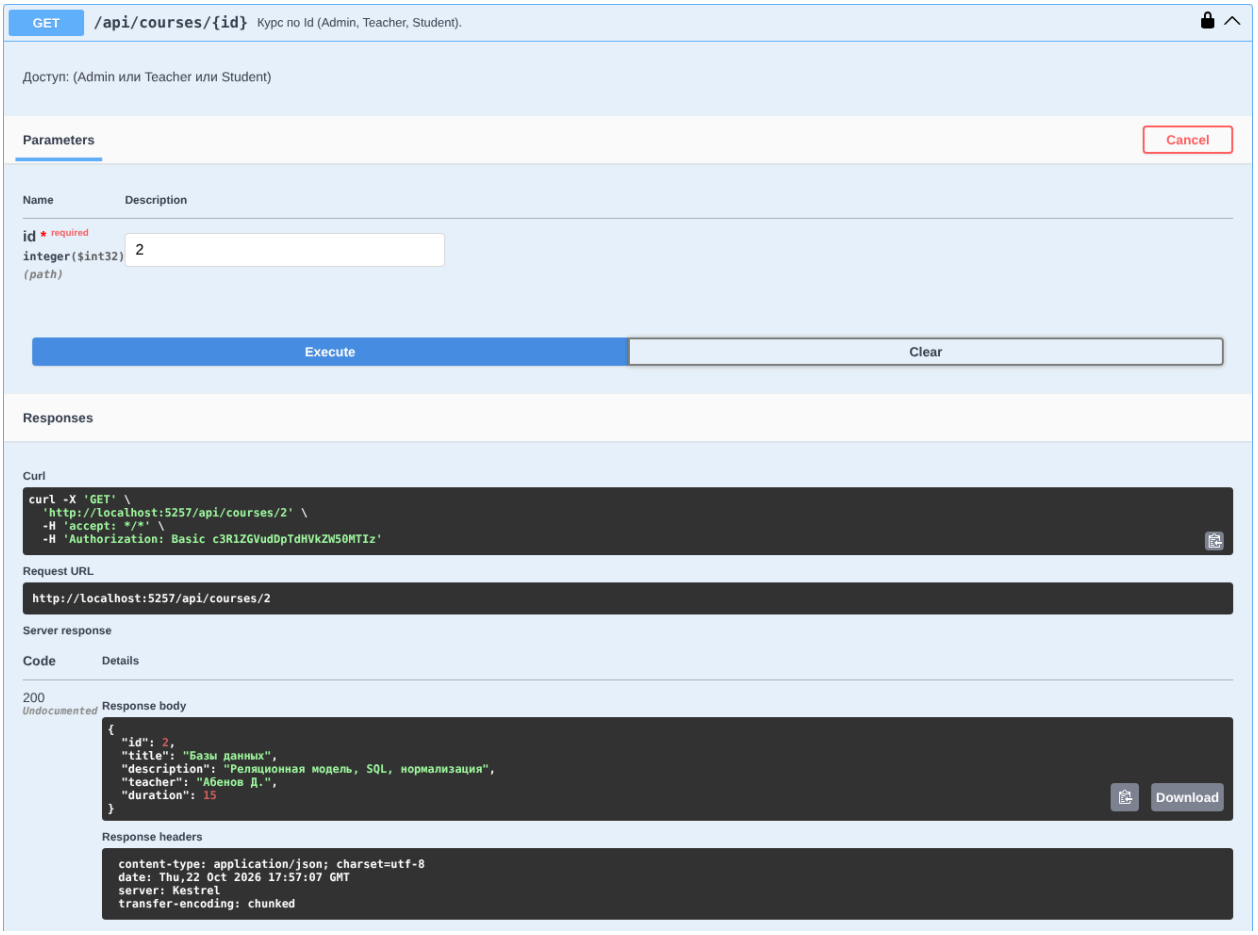
5 - Некорректная строка Base64 - 401 Unauthorized



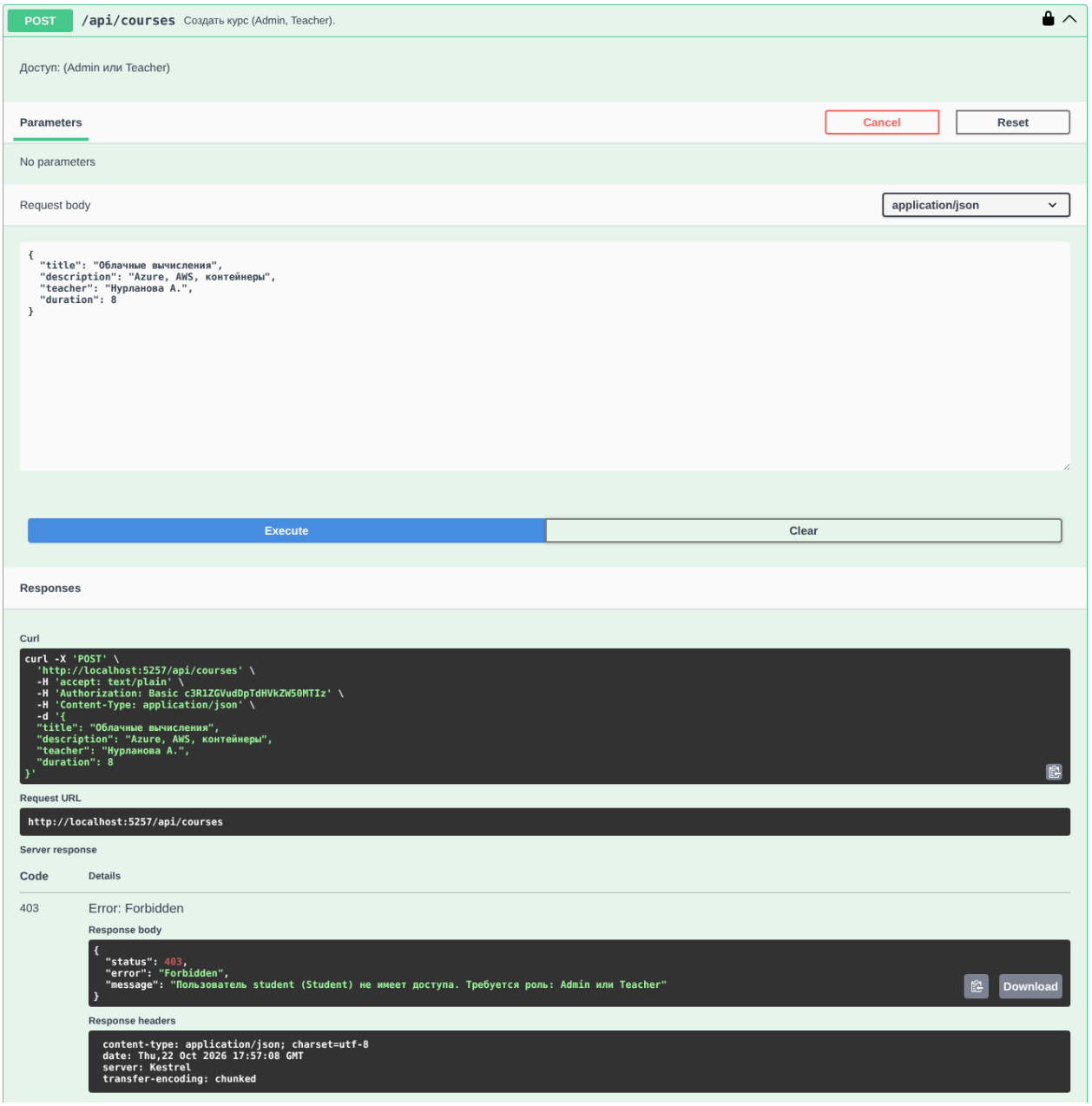
6 - Окно Authorize: student / Student123



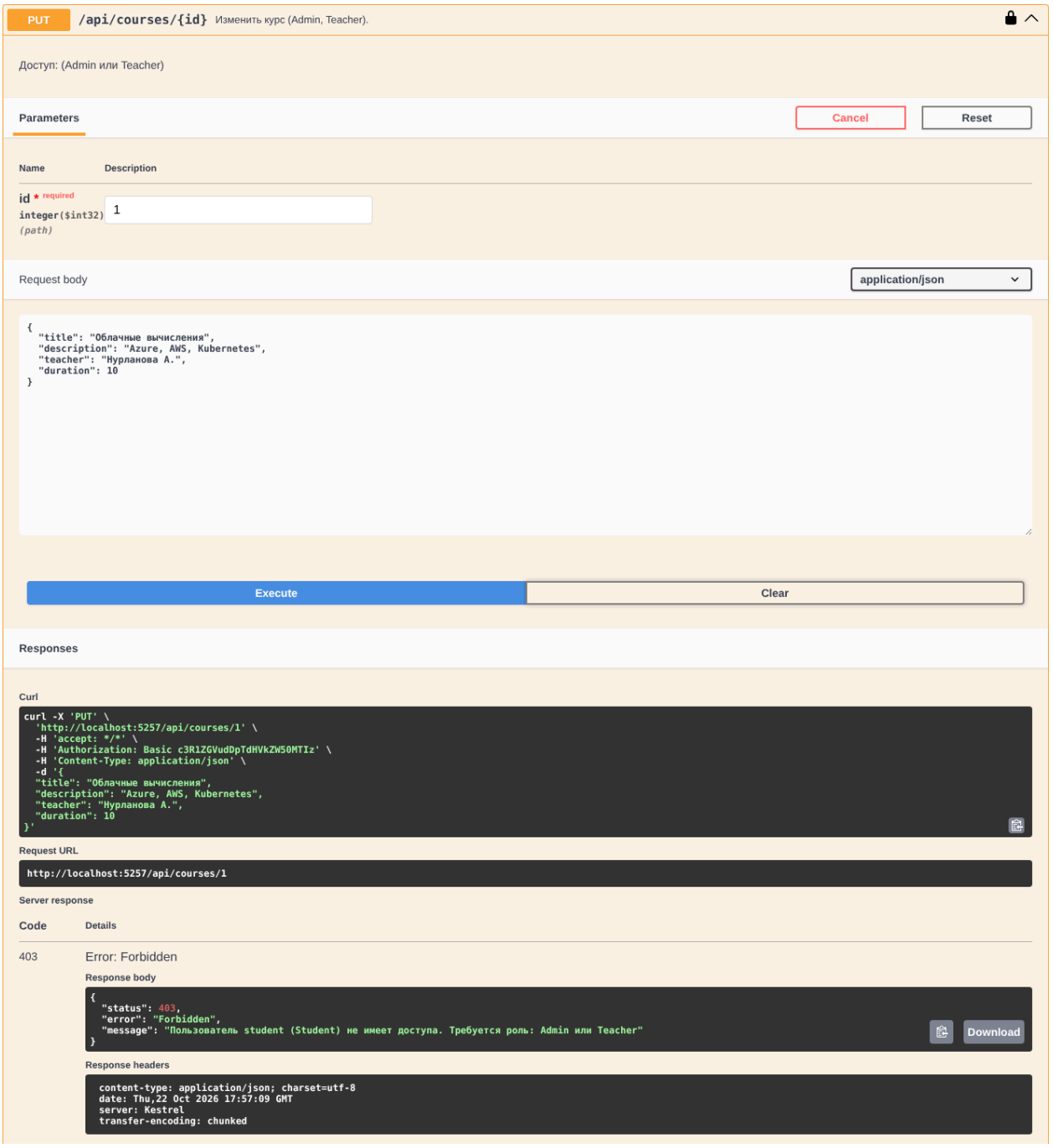
7 - Сценарий 2. Студент получает список курсов - 200 OK



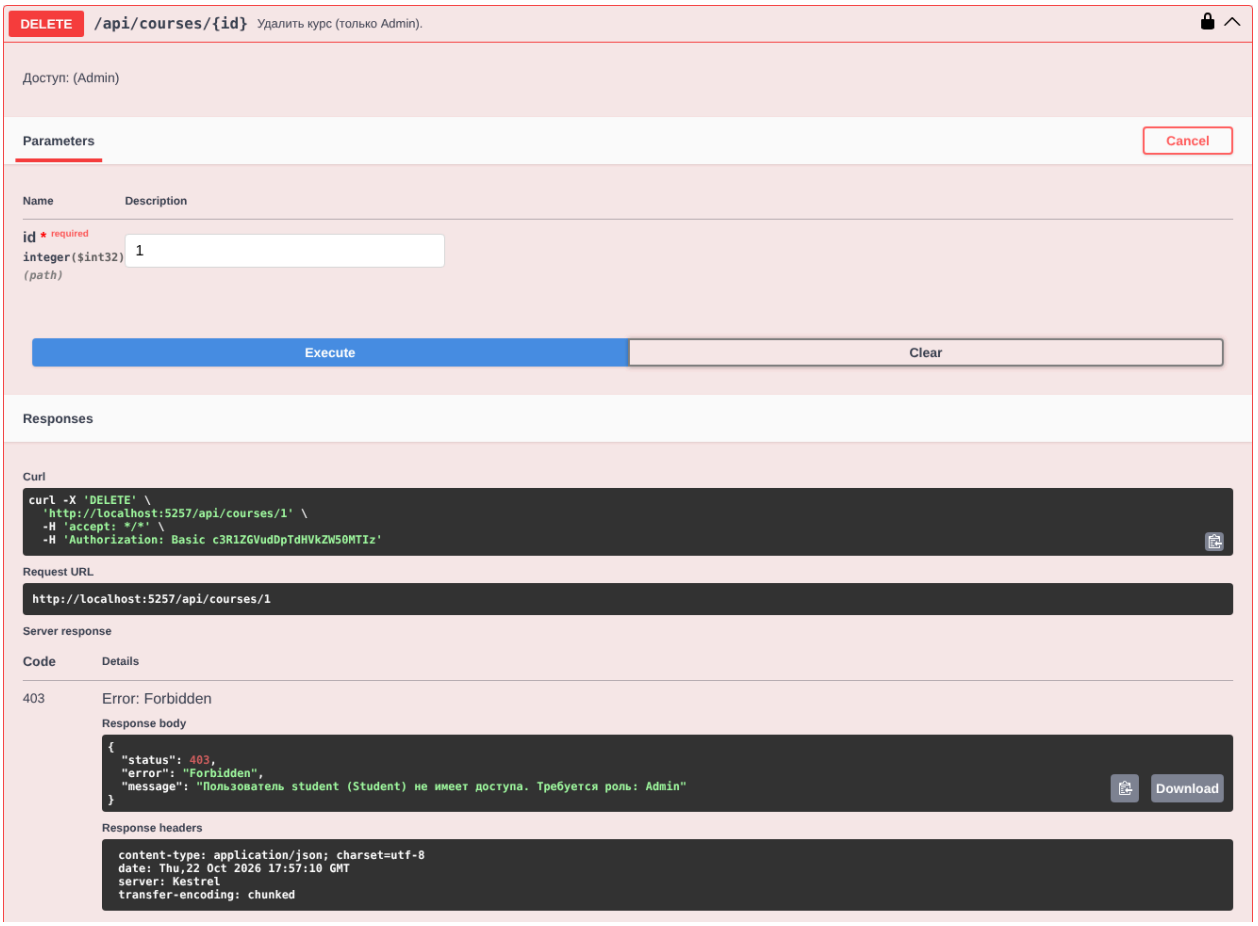
8 - Студент: GET /api/courses/2 - 200 OK



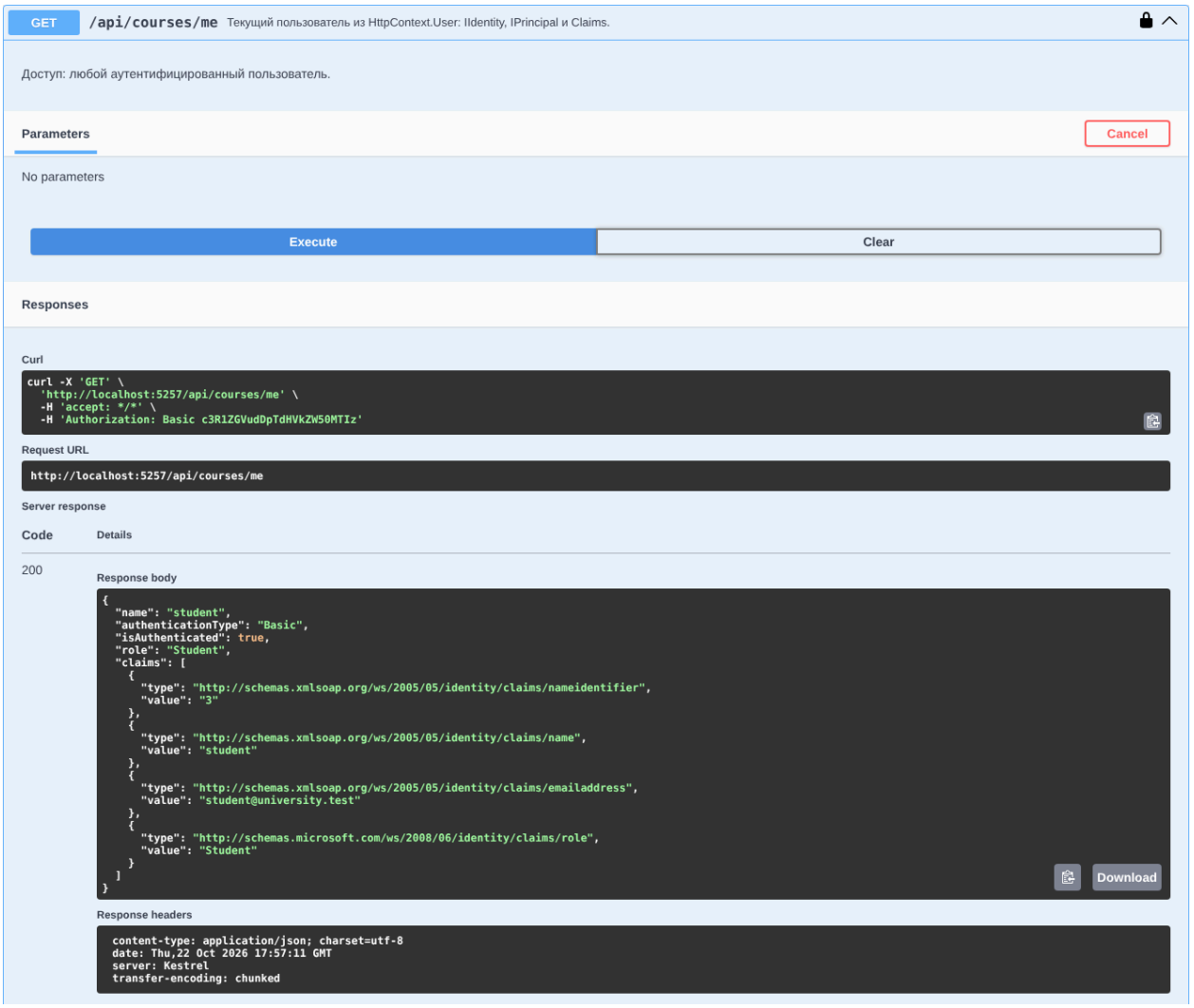
9 - Сценарий 4. Студент создаёт курс - 403 Forbidden



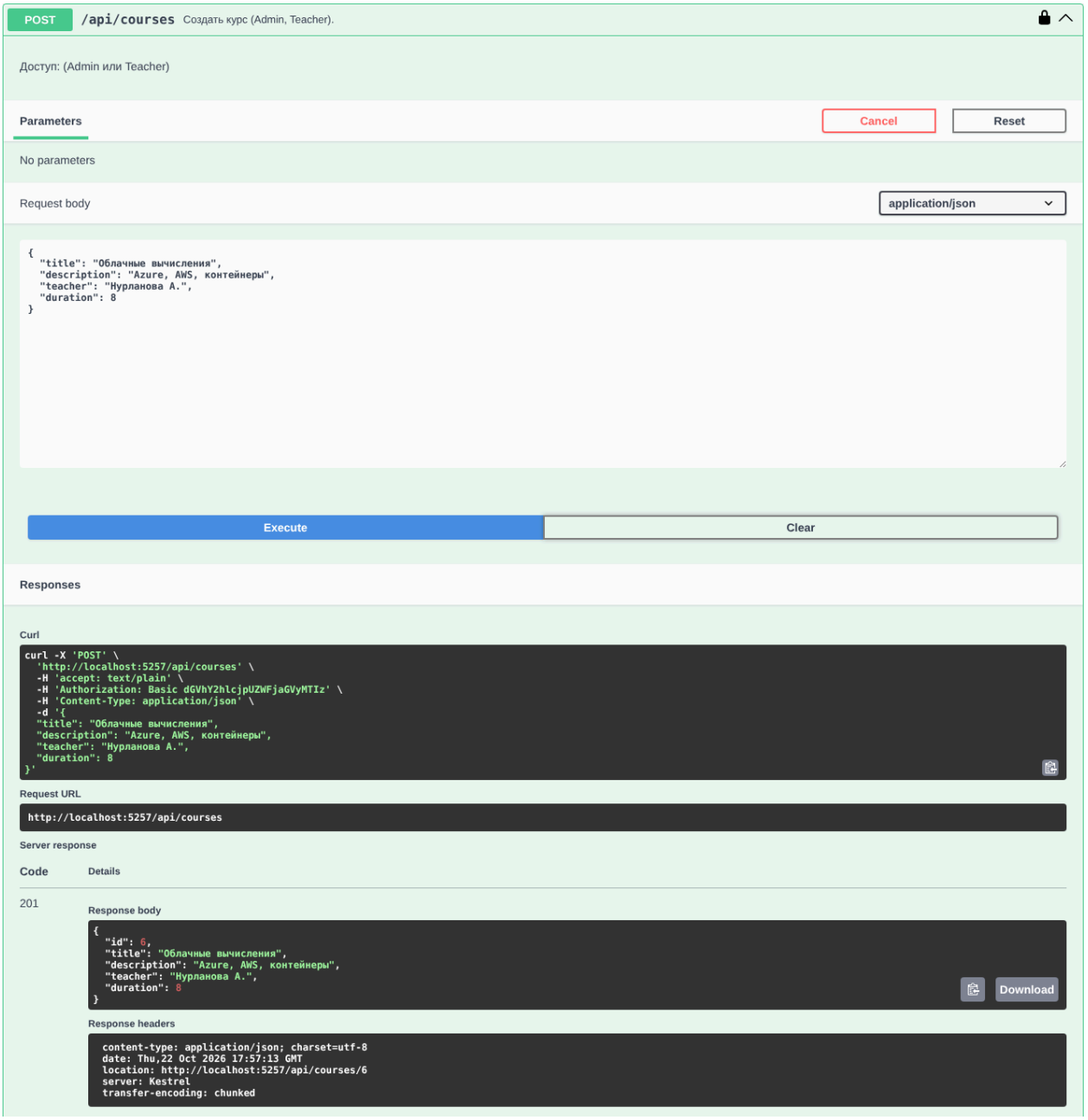
10 - Студент: PUT /api/courses/1 - 403 Forbidden



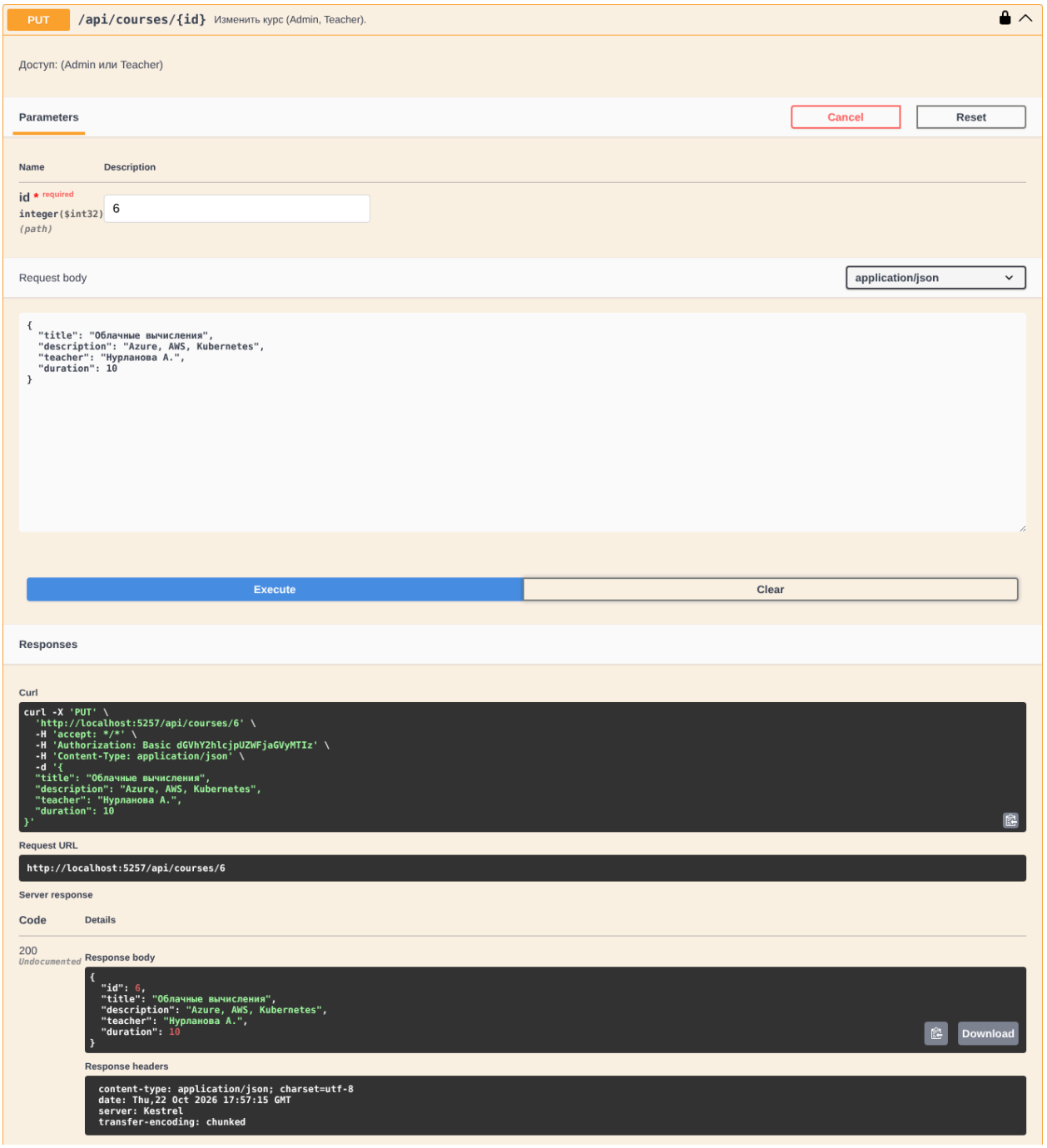
11 - Студент: DELETE /api/courses/1 - 403 Forbidden



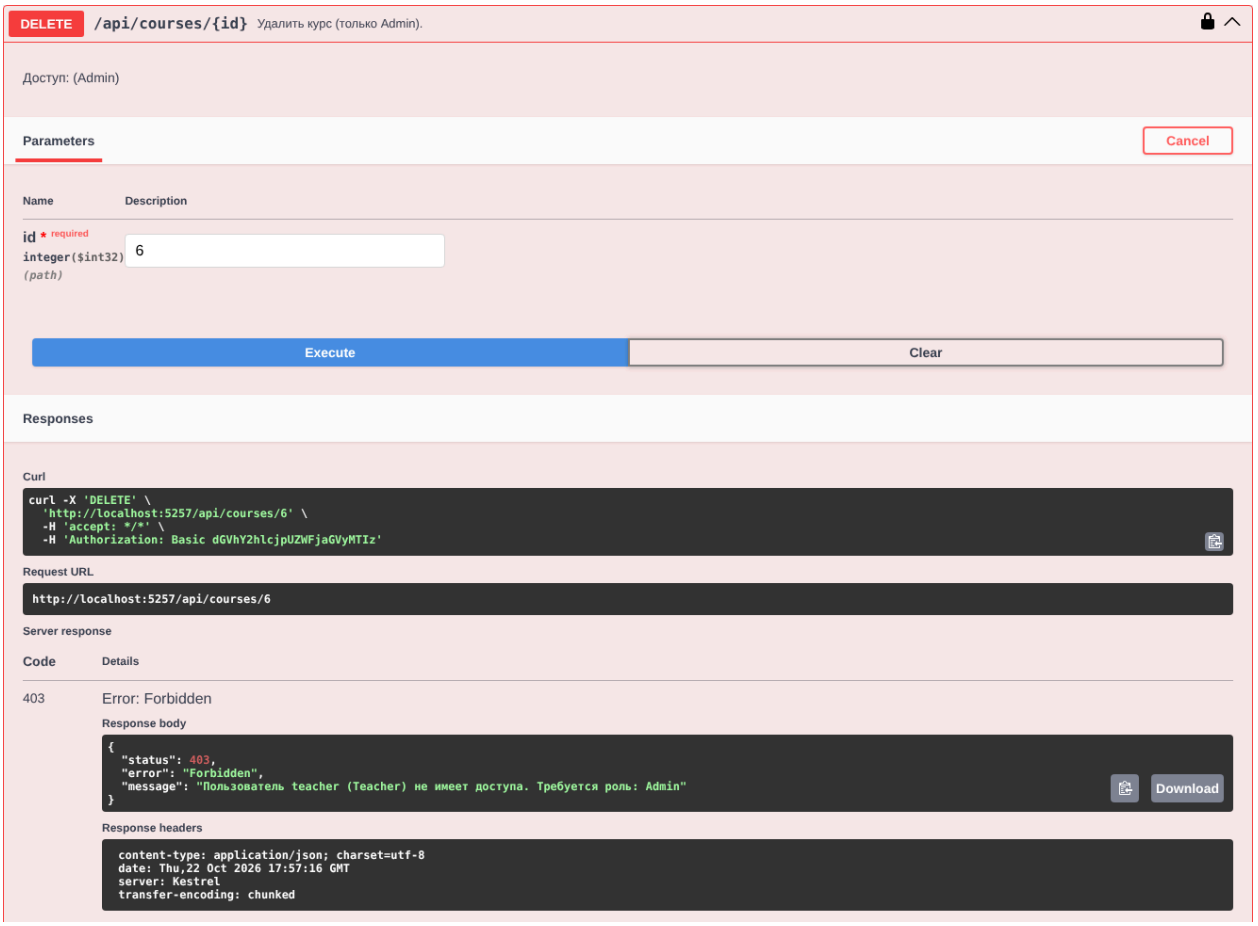
12 - Студент: GET /api/courses/me - IIdentity, IPrincipal и Claims текущего пользователя



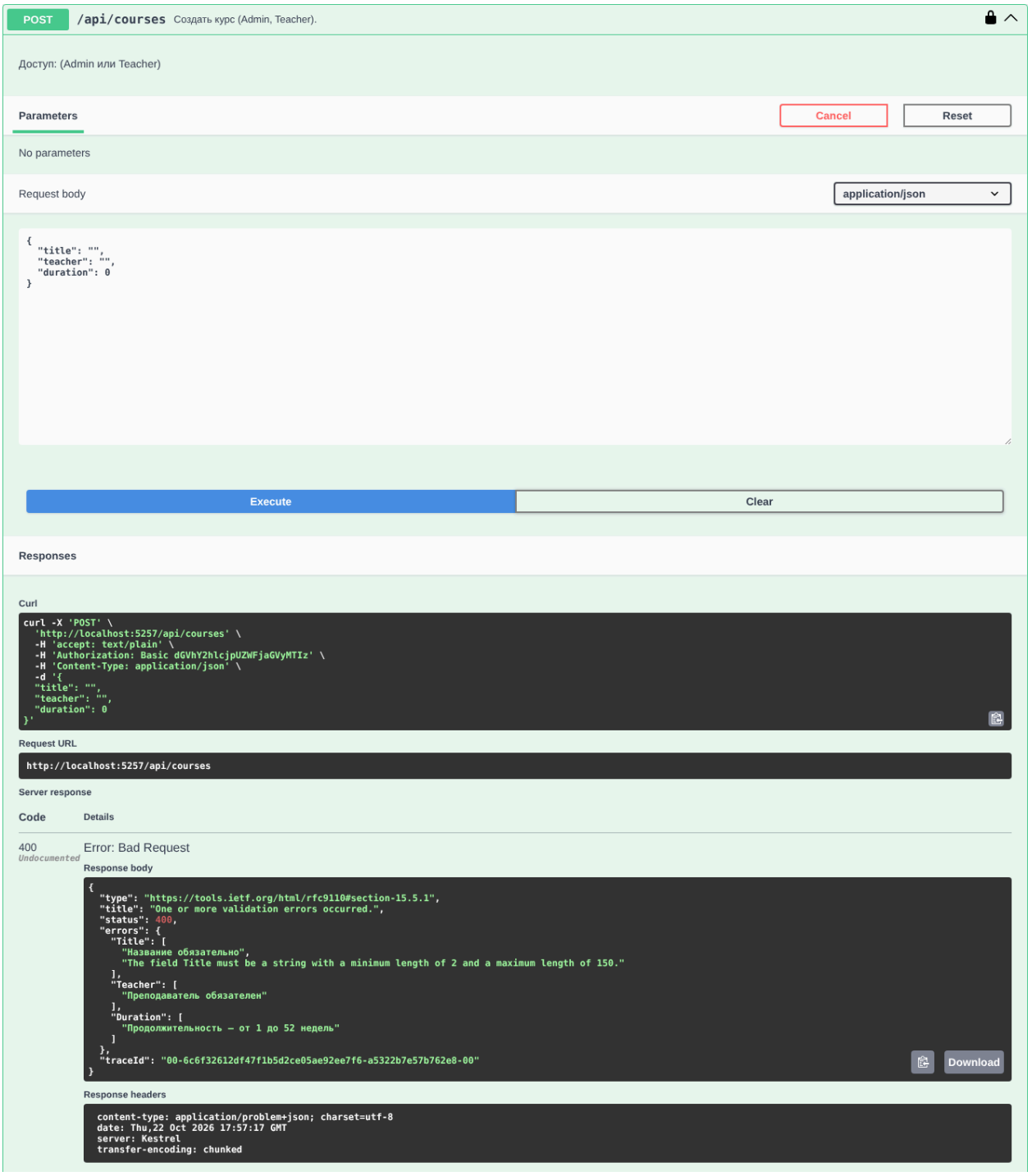
13 - Сценарий 3. Преподаватель создаёт курс - 201 Created



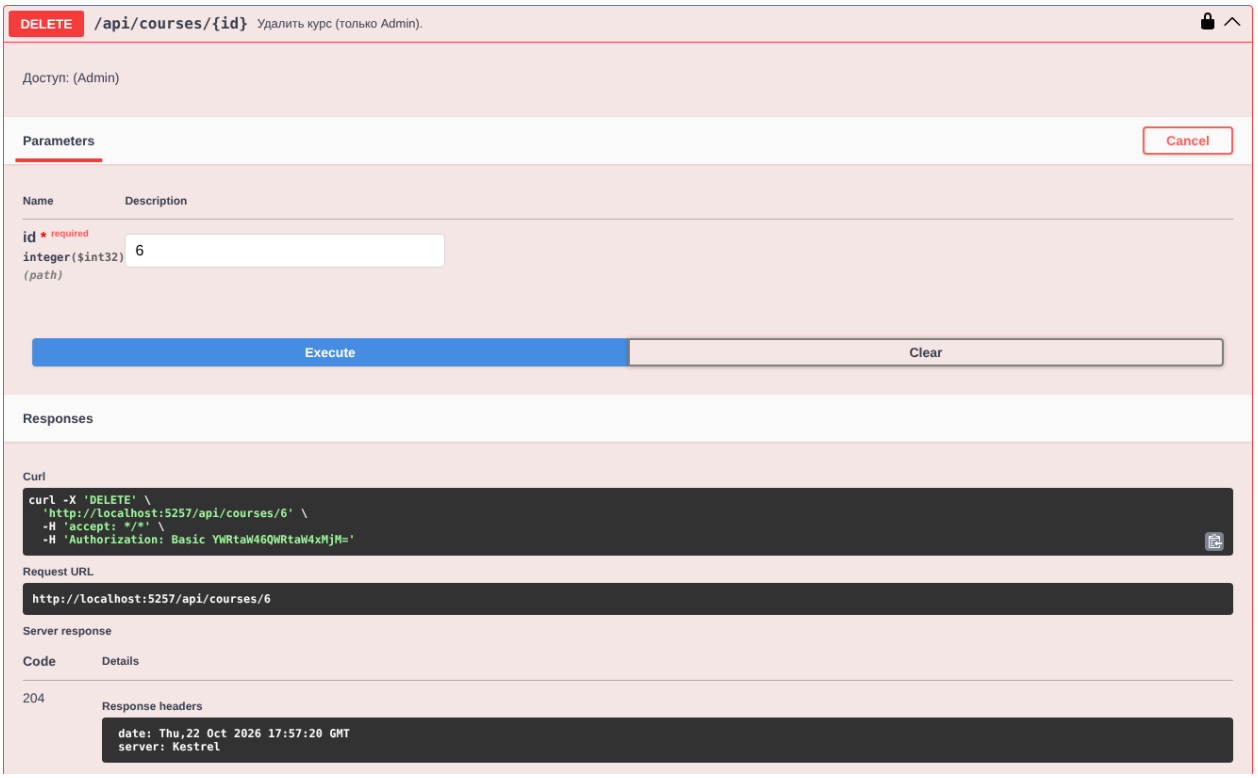
14 - Преподаватель: PUT /api/courses/6 - 200 OK



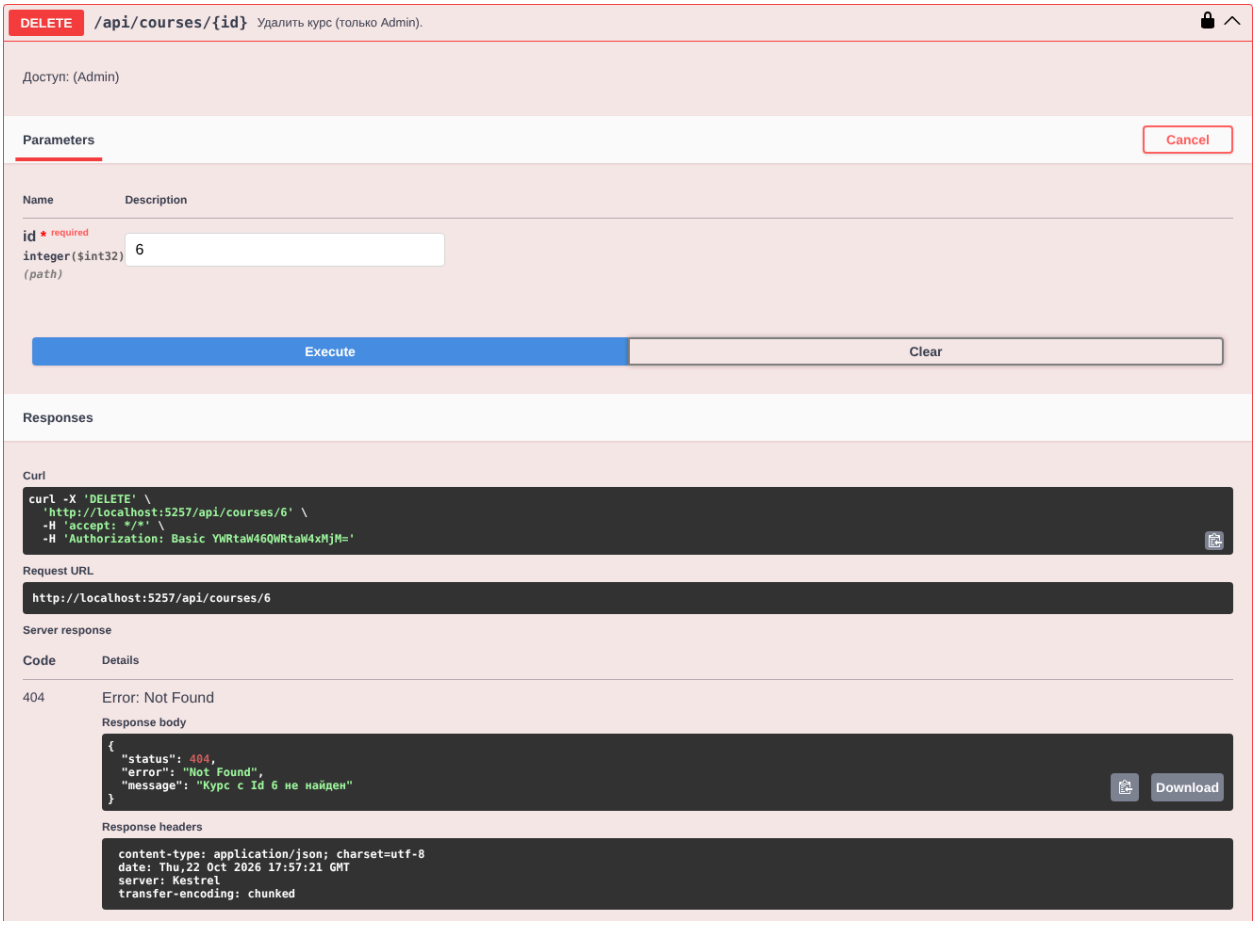
15 - Сценарий 6. Преподаватель удаляет курс - 403 Forbidden



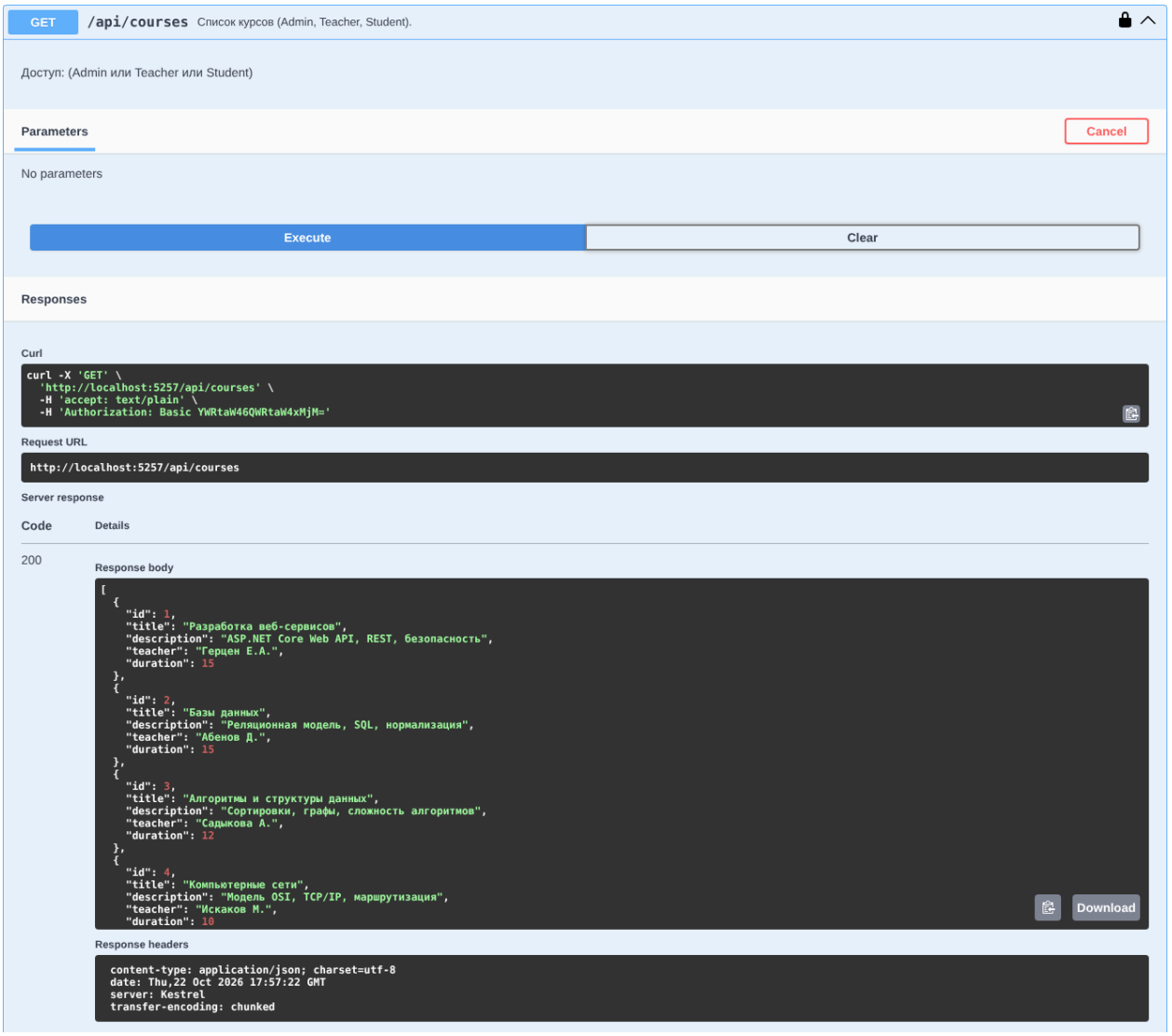
16 - Преподаватель: POST с ошибками в данных - 400 Bad Request



17 - Администратор удаляет курс - 204 No Content



18 - Администратор: повторное удаление 404 Not Found



19 - Администратор: GET /api/courses - 200 OK

Теоретические вопросы

1. Что такое аутентификация и авторизация?

Аутентификация - установление личности пользователя (кто вы?): проверка логина и пароля. Авторизация - проверка прав установленного пользователя (что вам можно?): проверка роли для конкретного метода. В API это два разных атрибута, выполняемых по очереди; ошибка первой даёт 401, второй - 403.

2. Как работает Basic Authentication?

Клиент отправляет в каждом запросе заголовок `Authorization: Basic <Base64(логин:пароль)>`, например `Basic c3R1ZGVudDpTdHVkZW50MTIz` для student:Student123. Сервер декодирует Base64, делит строку по первому «:», проверяет пользователя и устанавливает `HttpContext.User`. Если данных нет или они неверны, сервер отвечает 401 и заголовком `WWW-Authenticate: Basic realm="..."`. Сессий и токенов нет.

3. Для чего используются интерфейсы IIdentity и IPrincipal?

`IIdentity` описывает удостоверение: `Name`, `AuthenticationType`, `IsAuthenticated`. `IPrincipal` описывает пользователя целиком: свойство `Identity` и метод `IsInRole()`. Это общие абстракции .NET, поэтому код авторизации не зависит от способа аутентификации (Basic, JWT, cookies). Их реализации - `ClaimsIdentity` и `ClaimsPrincipal` (рисунок 12).

4. Что такое Claims и для чего они необходимы?

Claim - утверждение о пользователе в виде «тип — значение»: `name = student`, `email = student@university.test`, `role = Student`, `nameidentifier = 3`. Claims единообразно передают сведения о пользователе в код приложения: роли проверяются через `IsInRole`, остальные данные читаются через `User.FindFirst(...)`.

5. Чем отличаются ClaimsIdentity и ClaimsPrincipal?

`ClaimsIdentity`т- одно удостоверение, выданное одним способом аутентификации: набор Claims и `AuthenticationType`. `ClaimsPrincipal` - пользователь, у которого может быть несколько удостоверений (например, от Basic и от внешнего провайдера); он хранится в `HttpContext.User` и проверяет роли сразу по всем своим удостоверениям.

6. Как работает атрибут [Authorize]?

Стандартный `[Authorize]` добавляет к методу или контроллеру метаданные авторизации; middleware `UseAuthorization` перед вызовом метода проверяет, что `HttpContext.User` аутентифицирован, а также роли (`Roles = "Admin"`) или политики (`Policy = ...`). Нет пользователя - challenge (401), нет прав - forbid (403). В этой работе та же логика реализована вручную в `MyAuthorizeAttribute`.

7. Для чего используется [AllowAnonymous]?

Он разрешает доступ к методу без аутентификации, даже если контроллер защищён. Оба собственных атрибута проверяют наличие `IAllowAnonymous` в метаданных endpoint и пропускают такой запрос — поэтому `/api/courses/info` доступен анонимно (рисунок 3), а остальные методы контроллера требуют логин.

8. Чем отличаются HTTP-коды 401 Unauthorized и 403 Forbidden?

401 - «вы не представились или представились неверно»: нет заголовка, неправильный пароль, сломанный Base64; к ответу прилагается `WWW-Authenticate`. 403 — «вы известны, но вам нельзя»: student с верным паролем пытается создать курс. При 401 повтор с правильными данными поможет, при 403 - нет.

9. Какие недостатки имеет Basic Authentication?

Пароль передаётся в каждом запросе почти открыто (Base64 - не шифрование). Нет срока действия, выхода из системы и отзыва доступа без смены пароля. Сервер проверяет пароль при каждом запросе - дополнительная нагрузка. Браузер кэширует учётные данные. Нет защиты от подбора пароля и второго фактора. Поэтому Basic используется в основном для внутренних сервисов и только с HTTPS, а для публичных API выбирают токены (JWT, OAuth 2.0).

10. Почему для передачи учётных данных необходимо использовать HTTPS?

По HTTP заголовок `Authorization` идёт открытым текстом, и любой узел на пути (общий Wi-Fi, прокси) может его прочитать и раскодировать Base64 в логин и пароль. HTTPS шифрует весь запрос, включая заголовки, и подтверждает подлинность сервера сертификатом, защищая от подмены. В проекте есть профиль `https` и `app.UseHttpsRedirection()`.

Вывод

Разработан UniversityCoursesAPI с самостоятельно реализованной Basic-аутентификацией и авторизацией по ролям Admin, Teacher и Student. Учётные данные берутся из заголовка Authorization, декодируются из Base64 и проверяются в `UserService`; пользователь сохраняется в `HttpContext.User` как `ClaimsPrincipal`. `MyAuthorizeAttribute` ограничивает доступ к каждому методу; `[AllowAnonymous]` открывает публичную информацию. Все 8 сценариев задания дали ожидаемые коды: 401 без аутентификации и с неверным паролем, 403 при нехватке прав, 200/201/204 при разрешённых действиях.